TABLE OF CONTENTS

CineMatch AI — Full Project Blueprint

C

A

Collab

FastAPI

4
Project Phases

12+
ML Components

3
UI Layers

95%+
Target Accuracy

A production-grade, AI-powered movie recommendation system leveraging matrix factorization and cosine-similarity collaborative filtering. Delivers personalised Top-10 recommendations with Hit@K evaluation, deployed via a modern Next.js frontend and Gradio ML demo layer.

■■■ Sr. Developer

■ Sr. UX/UI Designer

■ Sr. ML Engineer

■ Sr. AI Engineer

■ Sr. Prompt Engineer

01 · PROJECT OVERVIEW & VISION

What we are building and why it matters

Project Identity

ATTRIBUTE	DETAIL
Project Name	CineMatch AI
Category	Machine Learning · Recommender Systems
ML Paradigm	Collaborative Filtering (User-User + Item-Item + Matrix Factorisation)
Primary UI	Next.js 14 (App Router) + Tailwind CSS
ML Demo Layer	Gradio 4.x (embedded in Next.js via iframe/API)
Backend API	FastAPI (Python 3.11)
Primary DB	PostgreSQL 16 + Redis 7 (recommendation cache)
Deployment	Docker Compose → AWS ECS Fargate + CloudFront CDN
Target Dataset	MovieLens 25M (25 million ratings, 62k movies, 162k users)
Evaluation KPI	Hit@10 ≥ 0.65 · NDCG@10 ≥ 0.42 · Coverage ≥ 40%

Problem Statement

Modern streaming platforms serve millions of titles yet most users discover only a fraction of content relevant to them. Generic popularity-based suggestions fail to capture individual taste. CineMatch AI solves this by learning latent preference patterns from historical ratings data — predicting not just what is popular but what a specific user will genuinely enjoy — and surfaces these as a ranked, explainable Top-10 list in under 200 ms.

Value Proposition

- Personalised Top-10 recommendations for any registered user in real time.
- Explainability layer — each recommendation shows 'Users like you also enjoyed...'.
- Cold-start handling via content-based fallback for new users (<5 ratings).
- A/B test framework baked in for continuous model improvement.
- Deployable as a standalone SaaS or embeddable widget for third-party streaming apps.

02 · ARCHITECTURE BLUEPRINT

System design — layers, services, data flows

High-Level Architecture Layers

LAYER	COMPONENT	TECHNOLOGY	RESPONSIBILITY
Presentation	Web App	Next.js 14 + Tailwind	User-facing UI, SSR, routing
Presentation	ML Demo	Gradio 4.x	Interactive ML exploration UI
API Gateway	REST API	FastAPI + Uvicorn	Auth, rate-limit, routing
ML Service	Rec Engine	Python + Surprise + SVD	Inference & ranking
ML Service	Similarity Svc	NumPy + Faiss	Cosine sim nearest-neighbours
Cache	Rec Cache	Redis 7	Pre-computed top-N recs
Database	User/Item DB	PostgreSQL 16	Users, items, ratings, logs
Storage	Model Store	AWS S3	Serialised model artefacts
Infra	Container Orch	Docker + AWS ECS	Scalable deployment
Monitoring	Observability	Prometheus + Grafana	Metrics, alerts, dashboards

Data Flow Narrative

1. User opens CineMatch AI → Next.js SSR page loads → Auth token validated by FastAPI Gateway.

2. Frontend calls GET /api/v1/recommendations/{user_id} → Gateway checks Redis cache (TTL 1 h).

3. Cache HIT → Return pre-ranked JSON list instantly (<20 ms). Cache MISS → invoke Rec Engine.

4. Rec Engine loads user-factor vector from PostgreSQL → runs SVD dot-product scoring against all items.

5. Top-N candidates passed through a post-filter (remove already-watched, low-confidence) → ranked list returned.

6. Result stored in Redis cache → returned to frontend → rendered as movie cards with similarity scores.

7. User interaction events (click, watch, rate) streamed back via POST /api/v1/events → logged to PostgreSQL.

8. Nightly batch job retrains SVD model on updated ratings → new model artifact pushed to S3 → hot-reloaded.

03 · TECHNOLOGY STACK DEEP-DIVE

Every tool, library and framework — with rationale

DOMAIN	TOOL / LIBRARY	VERSION	RATIONALE
Frontend UI	Next.js	14.2	App Router, SSR/ISR, SEO-friendly, React ecosystem
Frontend UI	Tailwind CSS	3.4	Utility-first, rapid design, dark-mode support
Frontend UI	Framer Motion	11	Smooth card animations, skeleton loaders
Frontend UI	SWR	2.2	Stale-while-revalidate data fetching, cache sync
ML Demo	Gradio	4.x	Rapid ML demo UI; chosen over Streamlit for richer compon
ML Demo	Gradio Themes	—	Custom dark theme matching brand
ML Core	Surprise	1.1	Production-tested SVD, NMF, KNN collaborative filters
ML Core	Implicit	0.7	ALS for implicit feedback (views, clicks)
ML Core	Faiss	1.7	GPU-accelerated nearest-neighbour search at scale
ML Core	NumPy / SciPy	1.26	Matrix ops, cosine similarity baseline
ML Core	Scikit-learn	1.4	Pipeline, metrics, calibration
API	FastAPI	0.111	Async, auto-docs (OpenAPI), Pydantic validation
API	Pydantic	2.x	Schema validation, serialisation
API	Redis-py	5.x	Cache client with connection pooling
Database	PostgreSQL	16	Relational integrity, JSONB for metadata
Database	SQLAlchemy	2.x	Async ORM, migrations via Alembic
Database	Redis	7.2	Caching, pub/sub for real-time events
DevOps	Docker / Compose	26	Reproducible local dev environment
DevOps	AWS ECS Fargate	—	Serverless containers, auto-scaling
DevOps	GitHub Actions	—	CI/CD pipeline: lint → test → build → deploy
Monitoring	Prometheus	2.x	Metrics collection (latency, hit rate, errors)
Monitoring	Grafana	10	Real-time dashboards for model and infra health

04 · DATA STRATEGY & PIPELINE

From raw MovieLens CSV to production-ready feature store

Dataset Profile — MovieLens 25M

ATTRIBUTE	VALUE
Ratings	25,000,095
Users	162,541
Movies	62,423
Rating Scale	0.5 – 5.0 (half-star increments)
Tag Records	1,093,360
Date Range	January 1995 – November 2019
Sparsity	~99.75% (typical for CF datasets)
Source	GroupLens Research (grouplens.org/datasets/movielens/25m/)

Data Pipeline Stages

STAGE	ACTION	OUTPUT	TOOL
01 · Ingest	Download & extract MovieLens 25M ZIP	Raw CSVs on disk	requests, wget
02 · Validate	Schema checks, dtype enforcement, null scan	Validation report JSON	Great Expectations
03 · Clean	Remove duplicate ratings, filter users < 20 ratings	Cleaned ratings.parquet	Pandas / Polars
04 · Feature Eng	Compute user RFM proxies, genre one-hot, tag TF-IDF	Feature matrix .parquet	Scikit-learn
05 · Split	Temporal 80/10/10 train/val/test split by timestamp	Split parquet files	Custom splitter
06 · Matrix Build	Construct sparse user-item rating matrix (CSR format)	Sparse CSR matrix	SciPy
07 · Normalise	Mean-centre user ratings; scale to [0,1]	Normalised matrix	NumPy
08 · Persist	Load into PostgreSQL ratings table; index on user_id	DB-ready data	SQLAlchemy bulk
09 · Cache Warm	Pre-compute Top-10 for top 10k active users → Redis	Redis hash store	redis-py + Celery

05 · ML/AI ENGINE DESIGN

Algorithms, training strategy, and inference pipeline

Algorithm Stack (3-Tier)

TIER	ALGORITHM	USE CASE	LIBRARY	ACCURACY TARG
Baseline	Global mean + bias	Cold-start fallback	NumPy	RMSE < 1.0
Tier 1	User-User KNN (cosine)	Sparse users (<50 ratings)	Surprise KNNBasic	Hit@10 > 0.50
Tier 2	Item-Item KNN (cosine)	Item similarity widget	Surprise KNNBasic	Hit@10 > 0.55
Tier 3	SVD (Matrix Factorisation)	Primary recommendation	Surprise SVD	Hit@10 > 0.65
Tier 4	ALS (Implicit Feedback)	Click/view data signal	Implicit ALS	Precision@10 > 0.45
Ensemble	Weighted blend SVD+ALS	Final production model	Custom blender	NDCG@10 > 0.42

SVD Hyperparameter Configuration

HYPERPARAMETER	SEARCH RANGE	OPTIMAL VALUE	TUNING METHOD
n_factors (latent dims)	50 – 300	128	Bayesian Optuna
n_epochs	10 – 60	30	Early stopping
lr_all (learning rate)	0.001 – 0.02	0.007	Grid search
reg_all (regularisation)	0.01 – 0.5	0.08	Grid search
biased	True / False	True	Ablation study

Cosine Similarity Baseline — Implementation Logic

Before deploying SVD, a cosine-similarity baseline is built to validate the data pipeline and establish a performance floor. The user-item sparse matrix is mean-centred per user (removing rating bias). sklearn.metrics.pairwise.cosine_similarity computes user-user similarity; top-K neighbours (K=40) are identified per user. A weighted average of neighbour ratings predicts missing entries. This baseline must achieve Hit@10 > 0.42 before SVD training begins — failing this signals data quality issues, not model issues.

Hit@K Evaluation — Exact Definition

Hit@K measures whether at least one item from the user's held-out test set appears in the model's top-K recommendations. For each user: generate top-K predictions; check intersection with ground-truth items (rating ≥ 4.0). A 'hit' is 1 if intersection is non-empty, else 0. Hit@K = mean(hits) across all test users. We target Hit@10 ≥ 0.65, Hit@20 ≥ 0.78. NDCG@10 weights position — a hit at rank 1 scores higher than a hit at rank 10.

06 · PHASE-BY-PHASE BUILD PLAN

12-week execution roadmap — sprints, tasks, and deliverables

PHASE 1 — Foundation & Data Infrastructure (Weeks 1-2)

Goal: Establish the full development environment, ingest MovieLens 25M, validate data quality, and confirm the database schema is production-ready.

Tasks

- Initialise monorepo: /frontend (Next.js), /backend (FastAPI), /ml (Python), /infra (Docker).
- Docker Compose: spin up PostgreSQL 16, Redis 7, Adminer, and PgAdmin containers.
- Write Alembic migrations: users, movies, ratings, genres, events, model_runs tables.
- Download MovieLens 25M; build ETL pipeline with Great Expectations validation suite.
- Load ratings.csv into PostgreSQL with COPY bulk-insert (< 90 s for 25 M rows).
- Create sparse CSR matrix from DB; persist as scipy .npz artefact to /ml/data/.
- Write unit tests for ETL pipeline (pytest); achieve 100% pass rate.
- Set up GitHub Actions CI: run lint (ruff, eslint) and pytest on every PR.

Deliverables

- ✓ Functional Docker Compose dev stack (docker compose up → all services healthy).
- ✓ 25 M ratings loaded in PostgreSQL (verified via COUNT query).
- ✓ Great Expectations validation report: 0 failed expectations.
- ✓ CSR matrix .npz file: shape (162541, 62423), sparsity ~99.75%.
- ✓ GitHub Actions CI green on main branch.

PHASE 2 — Cosine Similarity Baseline & Evaluation (Weeks 3-4)

Goal: Build, evaluate, and document the cosine-similarity collaborative filter baseline. This phase validates the data pipeline and establishes Hit@K floor metrics.

Tasks

- Implement mean-centred user-item matrix normalisation in /ml/preprocessing.py.
- Build UserUserCF class: fit(), predict(), recommend_top_k() methods.
- Build ItemItemCF class: item_similarity_matrix(), similar_items(item_id, k) methods.
- Temporal train/val/test split (80/10/10) by rating timestamp.
- Implement Hit@K, NDCG@K, Coverage, Diversity, Serendipity metrics in /ml/metrics.py.
- Run baseline evaluation: target Hit@10 > 0.42; log results to MLflow tracking server.
- Build Gradio demo v0.1: user_id input → Top-10 recommendation display.
- Write evaluation report markdown summarising baseline performance with failure analysis.

Deliverables

- ✓ UserUserCF and ItemItemCF classes with full docstrings.
- ✓ Baseline Hit@10 ≥ 0.42 (logged in MLflow; screenshot in report).
- ✓ Gradio demo v0.1 running on localhost:7860.
- ✓ Metrics module with 5 KPI functions (Hit@K, NDCG@K, Coverage, Diversity, Serendipity).
- ✓ Evaluation report PDF (auto-generated from Jupyter notebook via nbconvert).

PHASE 3 — SVD Matrix Factorisation Engine (Weeks 5-6)

Goal: Train, tune, and validate the production SVD model. Achieve Hit@10 ≥ 0.65. Integrate ALS for implicit signals. Build the ensemble blender.

Tasks

- Install Surprise; implement SVDModel class wrapping surprise.SVD with serialisation.
- Bayesian hyperparameter search with Optuna (100 trials, 4 GPU hours max).
- Train SVD with optimal params; serialize model to /ml/models/svd_v1.pkl via joblib.
- Train ALS model on implicit feedback (movies clicked/viewed, binary matrix).
- Build EnsembleRecommender: weighted blend (SVD 70% + ALS 30%, tunable).
- Run full evaluation suite on test set: SVD alone vs Ensemble.
- Implement Faiss IVF index over item factor vectors for sub-10 ms ANN search.
- Build RecommendationService class: get_recommendations(user_id, k=10) → ranked list.
- Push model artefacts to AWS S3 with versioning; document model card.

Deliverables

- ✓ SVD model: Hit@10 ≥ 0.65, NDCG@10 ≥ 0.42 (logged in MLflow).
- ✓ Ensemble model: marginal improvement over SVD alone (≥ 1% NDCG lift).
- ✓ Faiss index loaded: 10k user recommendation queries in < 5 s (500 QPS baseline).
- ✓ RecommendationService class with typed interface (Pydantic schemas).
- ✓ Model card documenting training data, metrics, limitations, bias analysis.

PHASE 4 — FastAPI Backend & Next.js Frontend (Weeks 7-9)

Goal: Build the production REST API and the full Next.js frontend. Wire Gradio demo as an embedded exploration tool within the web app.

Tasks

- Scaffold FastAPI app: /api/v1/recommendations, /api/v1/similar, /api/v1/events, /api/v1/search.
- JWT authentication (python-jose); rate limiting (slowapi); CORS configuration.
- Redis caching middleware: check cache → return / compute → store with 1-hour TTL.
- Async PostgreSQL access via SQLAlchemy 2.x + asyncpg driver.
- OpenAPI docs auto-generated at /docs; Postman collection exported.
- Next.js 14 app: pages — Home, Discover (rec feed), Movie Detail, Profile, Explore (Gradio).
- Design system: custom Tailwind config (dark theme, brand colours, typography scale).
- Movie card component: poster, title, year, genres, similarity score badge, hover effect.
- Recommendation feed: infinite scroll with SWR + optimistic updates.
- Gradio embed: iframe integration on /explore page with postMessage bridge for user_id.
- Responsive design: mobile-first, tested on 320px – 1920px viewports.
- Lighthouse score targets: Performance ≥ 90, Accessibility ≥ 95, SEO ≥ 90.

Deliverables

- ✓ FastAPI running on :8000; all 4 endpoints tested in Postman; 100% OpenAPI coverage.
- ✓ Redis cache operational: cache hit < 20 ms, cache miss < 250 ms (p95).

- ✓ Next.js app on :3000: Home → Discover → Movie Detail flow fully functional.
- ✓ Gradio demo embedded on /explore; user_id synced from auth context.
- ✓ Lighthouse scores: Performance 92, Accessibility 97, SEO 94.

07 · UX/UI DESIGN SYSTEM

Visual language, components, and user experience principles

Design Philosophy

CineMatch AI adopts a 'Cinematic Dark' design language — inspired by the atmosphere of premium streaming platforms (HBO Max aesthetic) combined with the data-forward clarity of analytics dashboards. The UI must feel immersive yet efficient: every click gets the user closer to their next favourite film. Microinteractions signal intelligence; the system feels alive and responsive.

Colour Palette

TOKEN	HEX	USAGE
--bg-primary	#0A0E1A	Page background, darkest layer
--bg-card	#111827	Movie cards, modal surfaces
--bg-elevated	#0D1A30	Section panels, sidebar
--accent-blue	#1A6CF6	Primary CTAs, links, active states
--accent-cyan	#00D4FF	Highlight scores, badges, data labels
--accent-purple	#7B2FFF	Genre tags, secondary actions
--accent-gold	#FFB800	Star ratings, premium tier badge
--text-primary	#F0F4FF	Headings, movie titles
--text-secondary	#8A93A8	Metadata, release year, runtime
--success	#00C896	Match score indicators
--border-subtle	#1A2A4A	Card borders, dividers

Typography Scale

ROLE	FONT	SIZE	WEIGHT	USE
Display	Inter	48px	800	Hero headline on landing page
Heading 1	Inter	32px	700	Page titles
Heading 2	Inter	24px	600	Section headers
Heading 3	Inter	18px	600	Card titles, movie names
Body Large	Inter	16px	400	Descriptions, synopses
Body	Inter	14px	400	Metadata, labels
Caption	Inter	12px	400	Tags, timestamps, fine print
Code / Score	JetBrains Mono	13px	500	Similarity scores, IDs

Key Screen Inventory

Landing / Hero: Animated movie backdrop, CTA to 'Get My Recommendations', login/signup.

Discover Feed: Infinite scroll grid of movie cards with match-score badges (0-100%).

Movie Detail: Full-bleed poster, synopsis, cast, 'Because you liked X' explanation row.

Explore (Gradio): Embedded Gradio panel for user_id → raw model output exploration.

Profile / History: Watch history, rating widget (half-star), preference sliders.

Search + Filter: Instant search with genre, decade, rating filters; similar movies sidebar.

Admin Dashboard: Model metrics (Hit@K trend), cache hit rate, active users (Prometheus).

08 · PROMPT ENGINEERING LIBRARY

Senior-grade prompts for every AI-assisted development task

All prompts below are crafted for Claude Sonnet / GPT-4o class models. They follow the RISEN framework: Role, Instructions, Steps, End-goal, Narrowing. Use them verbatim or adapt the [PLACEHOLDERS] for your specific context.

PROMPT 01 — Data Pipeline Architecture

```
You are a senior ML data engineer with 10 years of experience building recommendation
system pipelines.

TASK: Design a production-grade ETL pipeline for the MovieLens 25M dataset.

REQUIREMENTS:
- Input: Raw CSV files (ratings.csv 25M rows, movies.csv 62k rows, tags.csv)
- Output: Clean parquet files + PostgreSQL tables + scipy CSR sparse matrix
- Must handle: duplicate detection, user cold-start filtering (< 20 ratings),
timestamp-based splitting
- Include: Great Expectations validation suite with at least 8 expectations
- Performance target: full pipeline run < 15 minutes on 4-core CPU

OUTPUT FORMAT: Provide (1) architecture diagram in ASCII art, (2) Python code for
each stage as separate functions, (3) unit test stubs for each function, (4) a sample
Great Expectations checkpoint YAML config.

Do NOT use notebooks; all code must be modular .py files with type hints and
docstrings.
```

PROMPT 02 — SVD Model Training & Hyperparameter Tuning

```
You are a senior ML engineer specialising in matrix factorisation for recommender
systems.

TASK: Implement and tune an SVD collaborative filtering model for movie
recommendations.

CONTEXT:
- Dataset: MovieLens 25M (162k users, 62k movies, 25M ratings, scale 0.5-5.0)
- Library: Surprise 1.1 (do NOT use TensorFlow or PyTorch for this task)
- Tuning framework: Optuna with 100 trials

IMPLEMENTATION REQUIREMENTS:
1. SVDModel class with fit(train_data), predict(uid, iid), recommend_top_k(uid, k=10)
methods
2. Optuna study optimising RMSE on validation set; log all trials to MLflow
3. Hyperparameters to tune: n_factors [50-300], n_epochs [10-60], lr_all
[0.001-0.02], reg_all [0.01-0.5]
4. Early stopping callback if val RMSE does not improve for 5 epochs
5. Final model serialised with joblib; model card JSON saved alongside

TARGET METRICS: Hit@10 >= 0.65, NDCG@10 >= 0.42, RMSE <= 0.87

Provide complete, runnable Python code with inline comments explaining every design
decision.
```

PROMPT 03 — FastAPI Recommendation Endpoint

```
You are a senior backend engineer with expertise in async Python APIs and recommender
systems.

TASK: Build a production FastAPI endpoint for serving movie recommendations.

SPECIFICATIONS:
- Route: GET /api/v1/recommendations/{user_id}?k=10&strategy=svd
- Auth: Bearer JWT (python-jose); validate on every request
- Caching: Redis check-first pattern; TTL 3600s; key format
rec:{user_id}:{k}:{strategy}
- Cold-start: if user has < 5 ratings, fallback to popularity-based recommendations
- Response schema (Pydantic v2): {user_id, strategy, generated_at, recommendations:
[{movie_id, title, predicted_score, rank, explanation}]}
- Error handling: 404 if user not found, 429 if rate limit exceeded (100 req/min),
503 if model unavailable
- Logging: structured JSON logs (structlog) with request_id, user_id, latency_ms,
cache_hit

Also implement: POST /api/v1/events for logging user interactions (click, rate,
watch).
Include async SQLAlchemy for DB writes to avoid blocking the event loop.
Provide full code + pytest async test suite with mock Redis and DB fixtures.
```

PROMPT 04 — Next.js Recommendation Feed Component

```
You are a senior frontend engineer and UX specialist building a Netflix-quality movie
recommendation UI.

TASK: Build a React/Next.js recommendation feed for CineMatch AI.

TECH: Next.js 14 (App Router), TypeScript, Tailwind CSS 3.4, Framer Motion 11, SWR
2.2

DESIGN SYSTEM: Dark theme. Background #0A0E1A. Cards #111827. Accent blue #1A6CF6.
Highlight cyan #00D4FF.

COMPONENTS TO BUILD:
1. MovieCard: poster image (next/image), title, year, genres (pill tags), match-score
badge (0-100%, colour-coded: green >80, amber 60-80, red <60), hover lift + glow
effect
2. RecommendationFeed: SWR data fetching from /api/v1/recommendations/{userId},
infinite scroll, skeleton loading state (3 cards placeholder), error boundary with
retry
3. ExplainationTooltip: on hover over match score badge — show "Because you rated
[Movie X] 5 stars and [Movie Y] 4 stars"
4. EmptyState: shown for cold-start users — animated prompt to rate 10 seed movies

REQUIREMENTS: Fully typed TypeScript, WCAG AA accessibility (aria-labels, keyboard
navigation), Framer Motion stagger animation for cards entering viewport.
Mobile-first responsive (2 cols mobile, 4 cols desktop).

Deliver: complete component files, Tailwind config extensions, and Storybook stories
for each component.
```

PROMPT 05 — Gradio ML Demo Interface

```
You are a senior ML engineer and Gradio expert building an interactive recommendation
exploration tool.

TASK: Build a Gradio 4.x demo interface for CineMatch AI model exploration.

INTERFACE REQUIREMENTS:
Tab 1 — User Recommendations:
- Input: user_id (number), k (slider 5-50), strategy (radio: SVD / KNN / Ensemble)
- Output: DataFrame of top-K recs (rank, movie_id, title, genres, predicted_score,
explanation)
- Include: latency display, model version badge

Tab 2 — Item Similarity:
- Input: movie title (searchable dropdown from full catalog), k neighbours (slider)
- Output: similar movies DataFrame + cosine similarity scores (0-1)

Tab 3 — Model Diagnostics:
- Display: Hit@10 / NDCG@10 / Coverage trend chart (Plotly line chart)
- Input: date range picker → filter metrics history from MLflow API

STYLING: Use gr.themes.Base() with custom primary_hue='blue', neutral_hue='slate'.
Dark background matching brand (#0A0E1A).
DEPLOYMENT: Gradio share=False; exposed via FastAPI mount at /gradio path for
embedding in Next.js.

Provide complete app.py with error handling, loading states, and docstrings for every
function.
```

PROMPT 06 — Hit@K Evaluation Framework

```
You are a senior ML evaluation engineer and recommender systems researcher.

TASK: Implement a rigorous offline evaluation framework for ranking metrics.

METRICS TO IMPLEMENT (all as pure Python functions with NumPy, no external ranking
libs):
1. hit_at_k(recommended: List[int], relevant: Set[int], k: int) -> float — binary hit
2. ndcg_at_k(recommended: List[int], relevant: Set[int], k: int) -> float —
discounted gain
3. precision_at_k(recommended, relevant, k) -> float
4. recall_at_k(recommended, relevant, k) -> float
5. mrr(recommended, relevant) -> float — mean reciprocal rank
6. coverage(all_recommendations: List[List[int]], catalog_size: int) -> float
7. novelty(recommendations, item_popularity: Dict[int, int]) -> float
8. serendipity(recommendations, user_history, item_similarity_matrix) -> float

EVALUATION PIPELINE:
- evaluate_model(model, test_data, k_values=[5,10,20]) -> pd.DataFrame of results per
user
- aggregate_metrics(results_df) -> summary dict with mean, median, std per metric
- generate_report(summary) -> save PDF + JSON to /ml/reports/ with timestamp

ADDITIONAL: Implement statistical significance test (Wilcoxon signed-rank) comparing
two models.
Include complete pytest suite with deterministic test cases and expected values
documented.
```

09 · EVALUATION & METRICS FRAMEWORK

How we measure success at every layer of the system

ML Model KPIs

METRIC	FORMULA	BASELINE	TARGET	STRETCH
Hit@10	% users with ≥1 hit in top-10	0.42	0.65	0.72
Hit@20	% users with ≥1 hit in top-20	0.55	0.78	0.85
NDCG@10	Discounted hit position quality	0.28	0.42	0.50
Precision@10	Relevant / 10 on average	0.08	0.15	0.20
Recall@10	Hits / all relevant items	0.04	0.10	0.14
RMSE	Root mean sq error on ratings	1.05	0.87	0.82
MAE	Mean absolute error on ratings	0.82	0.68	0.63
Coverage	% catalog recommended to any user	12%	40%	55%
Novelty	Mean log-inverse popularity	3.2	4.5	5.0

System Performance SLAs

ENDPOINT	P50	P95	P99	ERROR RATE
GET /recommendations (cache hit)	8ms	20ms	35ms	< 0.1%
GET /recommendations (cache miss)	120ms	240ms	400ms	< 0.5%
GET /similar-items	15ms	40ms	80ms	< 0.1%
POST /events	5ms	15ms	30ms	< 0.1%
Gradio demo response	200ms	500ms	1000ms	< 1.0%

A/B Testing Framework

Two model variants run simultaneously (traffic split 50/50 via Redis feature flags). Control: SVD-only. Variant: Ensemble (SVD+ALS). Primary business metric: 7-day click-through rate on recommendations. Minimum detectable effect: 5% relative improvement. Statistical test: two-proportion z-test, alpha=0.05, power=0.80. Minimum experiment duration: 14 days. Decision framework: promote variant if CTR lift significant AND Hit@10 >= control.

10 · API SPECIFICATION

Full REST API contract — endpoints, schemas, and examples

METHOD	ENDPOINT	AUTH	DESCRIPTION
GET	/api/v1/recommendations/{user_id}	JWT	Get top-K recommendations for a user
GET	/api/v1/similar/{movie_id}	JWT	Get K most similar movies by item CF
GET	/api/v1/movies/search	JWT	Full-text search movies by title/genre
GET	/api/v1/movies/{movie_id}	None	Get movie metadata by ID
POST	/api/v1/events	JWT	Log user interaction event
POST	/api/v1/ratings	JWT	Submit or update a user rating
GET	/api/v1/users/me	JWT	Get current user profile
POST	/api/v1/auth/login	None	Exchange credentials for JWT
POST	/api/v1/auth/register	None	Register new user account
GET	/api/v1/admin/metrics	Admin	Model and system health metrics
POST	/api/v1/admin/retrain	Admin	Trigger async model retraining job

Response Schema — GET /api/v1/recommendations/{user_id}

```
{
  "user_id": 12345,
  "strategy": "ensemble",
  "generated_at": "2024-11-01T10:30:00Z",
  "cache_hit": false,
  "latency_ms": 187,
  "recommendations": [
    {
      "rank": 1,
      "movie_id": 318,
      "title": "The Shawshank Redemption",
      "genres": ["Drama"],
      "year": 1994,
      "predicted_score": 4.87,
      "match_percent": 97,
      "explanation": "Because you rated The Green Mile 5.0 and Se7en 4.5"
    }
  ]
}
```

11 · DEPLOYMENT & DEVOPS STRATEGY

From localhost to production AWS infrastructure

Deployment Architecture — AWS

COMPONENT	AWS SERVICE	CONFIG	SCALING
Next.js Frontend	CloudFront + S3	SSR via Lambda@Edge	Global CDN, auto
FastAPI API	ECS Fargate	2 vCPU, 4GB RAM per task	Auto-scale 2-20 tasks
Gradio Demo	ECS Fargate	1 vCPU, 2GB RAM per task	Fixed 2 tasks
PostgreSQL	RDS Aurora PostgreSQL	db.r6g.large, Multi-AZ	Read replicas
Redis Cache	ElastiCache Redis	cache.r7g.large, cluster mode	3-node cluster
Model Artefacts	S3 + S3 Transfer Acc	Versioned bucket	CDN-fronted
ML Training Jobs	AWS SageMaker	ml.m5.4xlarge spot instance	On-demand
Monitoring	CloudWatch + Grafana	Custom dashboards	Always-on
CI/CD	GitHub Actions	Build → ECR → ECS deploy	Per PR + main

CI/CD Pipeline — GitHub Actions

Step 1: PR opened → Lint (ruff + eslint) + type check (mypy + tsc) → must pass.

Step 2: Unit tests (pytest + jest) run in parallel; coverage report posted as PR comment.

Step 3: Docker images built for backend + ML service + frontend; pushed to AWS ECR.

Step 4: Integration tests run against ephemeral test stack (Docker Compose in CI).

Step 5: PR merged to main → Staging deploy to ECS (blue/green swap); smoke tests run.

Step 6: Manual approval gate → Production deploy; canary 5% traffic → 100% over 10 min.

Step 7: Post-deploy: Prometheus alerts watched for 15 min; auto-rollback if error rate > 1%.

12 · RISK REGISTER & MITIGATIONS

Known risks, likelihood, impact, and countermeasures

RISK	LIKELIHOOD	IMPACT	MITIGATION
Cold-start: new users have no ratings	HIGH	HIGH	Popularity-based fallback + onboarding flow (rate 1
Data sparsity degrades CF quality	HIGH	MEDIUM	Hybrid approach: content-based TF-IDF for sparse
SVD retraining stale: model drift over time	MEDIUM	HIGH	Nightly retrain job; Hit@K monitoring alert if drops
Redis cache invalidation complexity	MEDIUM	MEDIUM	TTL-based expiry (1h); event-driven invalidation or
MovieLens dataset not representative of real users	HIGH	MEDIUM	Document limitation in model card; test on real use
Gradio iframe CORS issues in Next.js	LOW	MEDIUM	Mount Gradio under FastAPI /gradio path; same or
AWS cost overrun during training	MEDIUM	LOW	SageMaker Spot instances; budget alerts at $50/$
Filter bubble: recommendations too similar	MEDIUM	MEDIUM	Diversity constraint: max 2 movies per genre in top
GDPR compliance for user ratings data	LOW	HIGH	Data anonymisation in export; right-to-delete endp

13 · TIMELINE & MILESTONES

12-week delivery roadmap with gates and checkpoints

WEEK	PHASE	MILESTONE	SUCCESS GATE
1	Phase 1	Repo + Docker stack initialised	docker compose up — all 4 services
2	Phase 1	MovieLens 25M loaded & validated	GE report 0 failures; 25M rows in PG
3	Phase 2	Cosine baseline trained	Hit@10 > 0.42 logged in MLflow
4	Phase 2	Gradio demo v0.1 live	Gradio on :7860; returns recs in <2s
5	Phase 3	SVD Optuna tuning complete	100 Optuna trials; best params save
6	Phase 3	Production SVD + Ensemble ready	Hit@10 >= 0.65 on test set
7	Phase 4	FastAPI all endpoints live	Postman collection 100% pass rate
8	Phase 4	Next.js core pages complete	Home + Discover + Detail functional
9	Phase 4	Gradio embedded; Lighthouse pass	Lighthouse perf >= 90; Gradio synce
10	Phase 4	Staging deploy on AWS	Blue/green staging deploy successfu
11	Hardening	Load test + bug fixes	500 RPS sustained; p95 < 250ms
12	Launch	Production go-live	All SLAs met; model card published

Post-Launch Roadmap (Weeks 13-20)

- Week 13-14: A/B test SVD vs Ensemble live traffic; measure CTR lift.
- Week 15: Implement collaborative filtering for implicit signals (ALS v2).
- Week 16: Add content-based hybrid layer using movie genre + overview TF-IDF.
- Week 17-18: Build 'Trending in your circle' social feature (friend graph via Neo4j).
- Week 19: NLP-based review sentiment integration for rating quality weighting.
- Week 20: Mobile app wrapper using React Native (shared API layer).

CineMatch AI — Full Project Blueprint v1.0 | Authored by: Sr. Developer · Sr. UX/UI Designer · Sr. ML Engineer · Sr. AI Engineer · Sr. Prompt Engineer | Ready for Phase 1 execution.